

Chapter 09: ACID Properties

Database Management System-এর অন্যতম গুরুত্বপূর্ণ বিষয় হলো **Transaction Management**। একটি Database-এ যখন কোনো Data Insert, Update বা Delete করা হয়, তখন সেই কাজকে অনেক সময় একটি **Transaction** হিসেবে বিবেচনা করা হয়।

ধরুন, একটি ব্যাংক অ্যাকাউন্ট থেকে **১০,০০০ টাকা** কেটে অন্য একটি অ্যাকাউন্টে পাঠানো হবে। এখানে দুটি কাজ করতে হবে—

1. প্রথম Account থেকে ১০,০০০ টাকা কমাতে হবে।
2. দ্বিতীয় Account-এ ১০,০০০ টাকা যোগ করতে হবে।

এখন প্রথম কাজটি সফল হলো কিন্তু দ্বিতীয় কাজটি ব্যর্থ হলো—তাহলে Database-এর Data ভুল হয়ে যাবে।

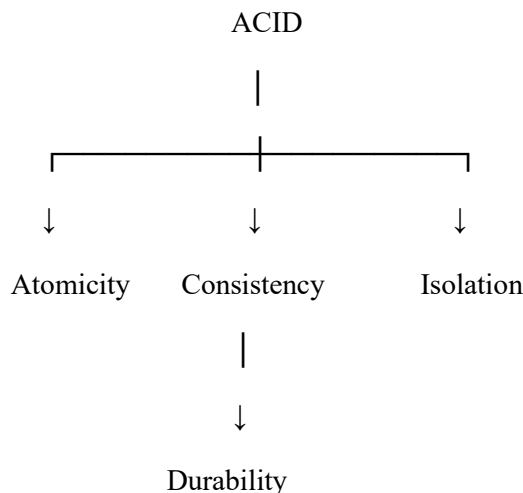
এই ধরনের সমস্যা থেকে Database-কে সুরক্ষিত রাখার জন্য Transaction-এর চারটি গুরুত্বপূর্ণ বৈশিষ্ট্য রয়েছে—

A — Atomicity
C — Consistency
I — Isolation
D — Durability

এই চারটি বৈশিষ্ট্যকে একসঙ্গে বলা হয় **ACID Properties**।

ACID Properties কী?

ACID Properties হলো Database Transaction-এর চারটি মৌলিক বৈশিষ্ট্য, যা Transaction-কে নির্ভরযোগ্য (Reliable), সঠিক (Correct) এবং নিরাপদ (Safe) রাখতে সাহায্য করে।



সহজভাবে মনে রাখার জন্য—

অক্ষর Property সহজ অর্থ

- A Atomicity সব হবে, না হলে কিছুই হবে না
- C Consistency Data-এর নিয়ম ও Integrity ঠিক থাকবে
- I Isolation Concurrent Transaction একে অপরকে অপ্রত্যাশিতভাবে প্রভাবিত করবে না
- D Durability Commit হওয়া Data স্থায়ী থাকবে

১. Atomicity

Atomicity কী?

Atomicity বলতে বোঝায়, একটি Transaction-এর সব Operation সফলভাবে সম্পন্ন হবে, অথবা কোনো Operation সফল হবে না—অর্থাৎ প্রয়োজন হলে পুরো Transaction Rollback হবে।

সহজভাবে—

Atomicity = All or Nothing

অর্থাৎ,

সব হবে, না হলে কিছুই হবে না।

Banking Example

ধরুন Rahim-এর Account থেকে Karim-এর Account-এ ১০,০০০ টাকা Transfer করা হবে।

Transaction:

Rahim Account

10,000 টাকা কমবে

↓

Karim Account

10,000 টাকা বাড়বে

ধরুন প্রথম কাজটি সফল হলো—

Rahim → -10,000

কিন্তু দ্বিতীয় কাজের সময় Error হলো।

যদি Atomicity না থাকে, তাহলে—

Rahim → টাকা কমেছে

Karim → টাকা পায়নি

এটি একটি ভুল Database State।

Atomicity নিশ্চিত করলে দ্বিতীয় Operation ব্যর্থ হওয়ার কারণে পুরো Transaction Rollback হবে।

Transaction Failed

↓

ROLLBACK

↓

পুরোনো Data ফিরে আসবে

SQL Server Example

BEGIN TRANSACTION;

UPDATE Accounts

SET Balance = Balance - 10000

WHERE AccountID = 101;

UPDATE Accounts

SET Balance = Balance + 10000

WHERE AccountID = 102;

COMMIT TRANSACTION;

যদি কোনো সমস্যা হয়, তাহলে—

ROLLBACK TRANSACTION;

এর মাধ্যমে পুরো Transaction বাতিল করা যায়।

২. Consistency

Consistency কী?

Consistency নিশ্চিত করে যে একটি Transaction সম্পন্ন হওয়ার পরে Database অবশ্যই তার নির্ধারিত Rules, Constraints এবং Integrity বজায় রাখবে।

সহজভাবে—

Consistency = Database সবসময় Valid State-এ থাকবে।

ধরুন একটি Table-এ একটি Account-এর Balance কখনো Negative হতে পারবে না।

Transaction-এর ফলে যদি Balance -5000 হয়ে যায় এবং Database-এর Business Rule তা অনুমোদন না করে, তাহলে Transaction সফল হওয়া উচিত নয়।

Consistency কী কী রক্ষা করে?

Database-এর বিভিন্ন Constraint এবং Rule Consistency বজায় রাখতে সাহায্য করে।

যেমন—

- Primary Key
- Foreign Key
- UNIQUE Constraint
- NOT NULL Constraint
- CHECK Constraint
- Data Type
- Business Rules

উদাহরণ

ধরুন—

```
CREATE TABLE Accounts
```

```
(
```

```
    AccountID INT PRIMARY KEY,
```

```
    Balance DECIMAL(12,2)
```

```
    CHECK (Balance >= 0)
```

);

এখানে Balance Negative হওয়া নিষিদ্ধ।

যদি কেউ লিখে—

```
UPDATE Accounts
```

```
SET Balance = -5000
```

```
WHERE AccountID = 101;
```

তাহলে Constraint-এর কারণে Database এই Invalid Data গ্রহণ করবে না।

এভাবেই Database-এর Integrity বজায় থাকে।

৩. Isolation

Isolation কী?

একই সময়ে একাধিক Transaction Database-এ কাজ করতে পারে।

Isolation নিশ্চিত করে যে একটি Transaction-এর Intermediate বা অসম্পূর্ণ পরিবর্তন অন্য Concurrent Transaction-এর কাছে অনাকাঙ্ক্ষিতভাবে দৃশ্যমান বা প্রভাবশালী না হয়।

সহজভাবে—

Isolation = Concurrent Transaction-গুলোকে নিয়ন্ত্রিতভাবে আলাদা রাখা।

বাস্তব উদাহরণ

ধরুন একটি ব্যাংক Account-এর Balance—

Balance = 50,000

একই সময়ে দুইটি Transaction চলছে।

Transaction A

Withdraw 20,000

Transaction B

Withdraw 40,000

যদি Database সঠিকভাবে Concurrent Transaction পরিচালনা না করে, তাহলে দুইটি Transaction একই পুরোনো Balance দেখে ভুল সিদ্ধান্ত নিতে পারে।

Isolation-এর উদ্দেশ্য হলো এই ধরনের Concurrent Access-এর কারণে Data Inconsistency কমানো।

Isolation Level

SQL Server-এ বিভিন্ন Isolation Level রয়েছে।

- READ UNCOMMITTED
- READ COMMITTED
- REPEATABLE READ
- SERIALIZABLE
- SNAPSHOT

প্রতিটি Isolation Level Consistency এবং Concurrency-এর মধ্যে ভিন্ন Trade-off তৈরি করে।

8. Durability

Durability কী?

Durability বলতে বোঝায়, কোনো Transaction সফলভাবে **COMMIT** হওয়ার পর সেই পরিবর্তন Database-এ স্থায়ী থাকবে, এমনকি পরে Server Crash বা System Restart হলেও।

সহজভাবে—

Durability = Commit হলে Data হারাবে না।

Banking Example

ধরুন একজন Customer ১০,০০০ টাকা Deposit করলেন।

Transaction সফল হলো—

BEGIN TRANSACTION

↓

Deposit 10,000

↓

COMMIT

এখন যদি Server Restart হয়, তাহলেও Deposit-এর পরিবর্তন হারিয়ে যাওয়া উচিত নয়।

Database-এর Recovery Mechanism এবং Transaction Log-এর মতো ব্যবস্থাগুলো এই Durability নিশ্চিত করতে গুরুত্বপূর্ণ ভূমিকা পালন করে।

ACID-এর একটি সম্পূর্ণ উদাহরণ

ধরুন একটি Online Shopping System-এ Customer একটি Product Order করল।

Order করার সময় কয়েকটি কাজ হতে পারে—

1. Order তৈরি

↓

2. Product Stock কমানো

↓

3. Payment Record তৈরি

↓

4. Order Status Update

↓

5. Transaction Commit

এখানে ACID কীভাবে কাজ করে?

Atomicity

সব Operation সফল হবে, অথবা Transaction Rollback হবে।

Consistency

Stock Negative হবে না এবং Database-এর Constraints বজায় থাকবে।

Isolation

একই Product একই সময়ে অনেক User Order করলে Concurrent Transaction সঠিকভাবে পরিচালিত হবে।

Durability

Order Commit হওয়ার পরে Server Restart হলেও Order-এর Data থাকবে।

ACID মনে রাখার সহজ পদ্ধতি

A — Atomicity

All or Nothing

সব হবে, না হলে কিছুই হবে না।

C — Consistency

Correct State

Transaction-এর আগে ও পরে Database-এর Rules ও Integrity বজায় থাকবে।

I — Isolation

Independent Execution

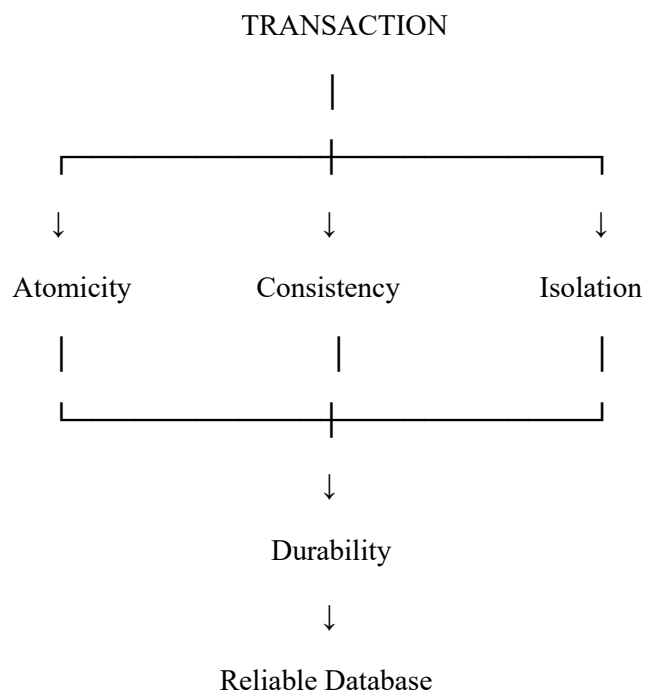
Concurrent Transaction একে অপরের Intermediate State-কে অনাকাঙ্ক্ষিতভাবে প্রভাবিত করবে না।

D — Durability

Don't Lose Committed Data

Commit হওয়া Data স্থায়ী থাকবে।

ACID-এর চারটি বৈশিষ্ট্য একসঙ্গে



ACID কেন গুরুত্বপূর্ণ?

ACID Properties ছাড়া গুরুত্বপূর্ণ Database Application-এ Data Corruption, Inconsistency এবং Transaction Failure-এর ঝুঁকি বেড়ে যেতে পারে।

বিশেষ করে—

- Banking System
- Payment Gateway
- E-Commerce
- Accounting System
- ERP
- Inventory Management
- Hospital Management
- Financial Application

এর মতো System-এ Transaction-এর Reliability অত্যন্ত গুরুত্বপূর্ণ।

ACID বনাম CAP

ACID এবং CAP Theorem একই বিষয় নয়।

ACID মূলত Database Transaction-এর Reliability এবং Correctness নিয়ে আলোচনা করে।

অন্যদিকে CAP Theorem Distributed System-এ Consistency, Availability এবং Partition Tolerance-এর Trade-off নিয়ে আলোচনা করে।

সহজভাবে—

ACID

↓

Transaction Reliability

CAP

↓

Distributed System Trade-off

অধ্যায়ের সারসংক্ষেপ

ACID Properties হলো Database Transaction-এর চারটি মৌলিক বৈশিষ্ট্য—**Atomicity, Consistency, Isolation এবং Durability**।

Atomicity নিশ্চিত করে Transaction হবে সম্পূর্ণভাবে, অথবা প্রয়োজনে সম্পূর্ণ Rollback হবে।

Consistency নিশ্চিত করে Transaction-এর পরে Database Valid State-এ থাকবে। **Isolation** Concurrent Transaction-গুলোর মধ্যে অনাকাঙ্ক্ষিত প্রভাব নিয়ন্ত্রণ করে। আর **Durability** নিশ্চিত করে Commit হওয়া Data পরবর্তীতে Server Failure বা Restart হলেও স্থায়ী থাকবে।

সুতরাং,

A = All or Nothing

C = Correct Data State

I = Independent/Controlled Transactions

D = Don't Lose Committed Data

এই চারটি বৈশিষ্ট্য একসঙ্গে একটি Database Transaction-কে **Reliable, Consistent এবং Trustworthy** করে তোলে।